

STUDIOFLOW

AI-Powered Operating System
for Photography Studios

PROJECT SPECIFICATION BOOK

University Software Project

STUDIOFLOW

AI-Powered Operating System for Photography Studios

Groupe Members:

Mohamed Yessine Nahdi

Amel Mediouni

Louay Herichi

Yosser Rafrafi

Aya Flah

Anas Ben Ibrahim


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

SUMMARY

01. Project Presentation

1.1 General Context
1.2 Problematic
1.3 Study of Existing Solutions
1.4 Competitor Analysis
1.5 Proposed Solution

02. Requirements Analysis

2.1 Functional Requirements
2.2 Non-Functional Requirements
2.3 Actors
2.4 Team Organization

03. Graphic Design

3.1 Graphic Charter

04. Technical Conception

4.1 Technology Stack
4.2 Software Architecture
4.3 Class Diagram
4.4 Data Modeling (MongoDB)



01

Project Presentation

1.1 GENERAL CONTEXT

A concise view of the operational reality StudioFlow is designed to transform.

The fragmented studio day

Photography studios coordinate clients, projects, schedules, employees, photographs, documents, editing, selections, and delivery across disconnected tools. This fragmentation creates repetitive work, scattered information, and limited visibility from one stage to the next.

The opportunity

AI makes it possible to analyze photographs, organize archives, support selections, coordinate schedules, interpret documents, and accelerate image processing. Used thoughtfully, it can reduce operational friction without replacing creative judgment.

The StudioFlow response

StudioFlow is a centralized operating system for the modern photography studio. It unifies the workflow from capture to delivery, bringing people, projects, assets, decisions, and next actions into one coherent operating layer.

The studio workflow

A continuous operating flow connecting the essential stages of studio work.

Capture	Organize	Select	Edit	Deliver
Bring the work into the studio.	Create order and context.	Make decisions with clarity.	Process images efficiently.	Share the final result.

1.2. Problematic

Photography studios rely on multiple tools to manage photos, clients, projects, employees, editing, and file delivery. This fragmentation makes workflows more complex, time-consuming, and difficult to coordinate.

How can we create a unified platform that centralizes photography-studio operations while using AI to automate repetitive tasks, improve photo management, assist clients and employees, and optimize the overall workflow?

1.3 STUDY OF EXISTING SOLUTIONS

Four solution categories shape the current landscape — each solves a valuable part of the workflow, but none connects the complete studio operating model.

01 Photography management platforms
Studio workflows, bookings and client records.
Typical strengths — Centralized studio administration and client coordination.
Typical limitations — Often limited in AI depth or specialized image intelligence.

02 Online gallery platforms
Polished delivery, proofing and client selection.
Typical strengths — Clear presentation and smooth client-facing review.
Typical limitations — Often separate from internal staffing, project and document workflows.

03 AI photo editing tools
Efficient enhancement, retouching and style processing.
Typical strengths — Faster image refinement and repeatable visual treatment.
Typical limitations — Often focused on image output rather than studio operations.

04 General project and collaboration tools
Flexible tasks, files and teamwork.
Typical strengths — Adaptable coordination across people and projects.
Typical limitations — Often lack photography-specific galleries, image intelligence and proofing.

1.4 COMPETITOR ANALYSIS

A capability view of the operating space around image-led studio work.

Capability	Studio management platform	Online gallery platform	AI photo tool	General collaboration tool	StudioFlow
Studio Management	Core	Limited	External	External	Integrated
Client Gallery	Focused	Core	External	Limited	Integrated
Photo Organization	Focused	Focused	Core	Limited	Integrated
AI Photo Features	Limited	Limited	Core	External	Integrated
Sessions & Equipment	Core	External	External	Focused	Integrated
Billing, Portfolio & Reviews	Core	Focused	External	Limited	Integrated

TAKEAWAY

StudioFlow is designed around the intersections between image intelligence, people, clients and sessions.

1.5 PROPOSED SOLUTION

StudioFlow is a centralized AI-powered platform that brings the studio's creative operations, people, clients, assets and delivery workflows into one clear operating model.

Seven modules: one shared core, six business modules

Each module addresses a distinct operational need while sharing the same data, permissions and studio context. Each business module is owned by one team member.

CORE Gestion des Utilisateurs

Accounts, authentication, roles (RBAC) and notifications shared by every module.

01 Gestion des Séances

Shoot planning, locations, photographer assignment and AI-suggested time slots.

02 Gestion des Photos & Galeries

Bulk upload, AI culling and tagging, smart search, private client galleries and selection.

03 Gestion du Matériel

Equipment inventory, reservations per session, returns and breakdown tracking.

04 Gestion de la Facturation

AI-generated quotes, deposit and balance invoices, payment follow-up.

05 Gestion du Portfolio

Public showcase of the studio's best work, curated and captioned with AI.

06 Gestion des Avis & Réclamations

Client reviews with AI sentiment analysis, complaint handling and prioritization.

Shared infrastructure

The modules are connected by a common foundation that keeps information secure, discoverable and actionable across the studio.

AUTHENTICATION · RBAC · CLIENT PORTAL · CLOUD STORAGE · MONGODB DATABASE · NOTIFICATIONS · AI SERVICES · SHARED UI · APIs · DEPLOYMENT

Seven modules, one operating model.

END-TO-END STUDIO WORKFLOW

THE CONTINUOUS PATH

01	02	03	04	05	06	07	08
Session Booking →	Quote & Deposit →	Equipment Booking →	Shoot & Upload →	AI Culling →	Client Selection →	Invoice & Delivery →	Review & Portfolio

ONE CONTINUOUS PATH
StudioFlow connects sessions, image intelligence, client decisions, billing and feedback into one continuous path.

Human-in-the-loop
AI accelerates discovery and preparation; studio teams retain final creative and operational control.

02

Requirements Analysis

Functional scope, quality attributes and modular delivery



2.1 FUNCTIONAL REQUIREMENTS · CORE PLATFORM

This section defines the core platform capabilities required to support StudioFlow as a connected, modular operating system for creative studios.

Area	Required capabilities
Gestion des Utilisateurs (Authentication / Users)	Secure sign-in, user accounts, profiles and administration. Shared core module.
Role-Based Access Control	Four roles (Admin, Photographer, Retoucher, Client) checked on every route and service.
AI Services Layer	Shared AI clients and background jobs used by all modules, with status Pending → Processing → Completed / Failed.
Storage	Cloud storage (Cloudinary) for photos and documents; MongoDB keeps only metadata and URLs.
Client Portal	Controlled client access to sessions, quotes, invoices, galleries, selections and reviews.
Notifications	Event-driven updates for sessions, payments, gallery sharing and complaints.
API / Module Communication	Next.js Route Handlers and Server Actions; modules talk only through their service layer.
Testing / Module Independence	Each module must be independently testable, with its own services and stable interfaces.

2.1 FUNCTIONAL REQUIREMENTS · BUSINESS MODULES

Each business module is owned by one team member and covers its entities end to end (data model, services, API and interface).

Module	Required capabilities
Gestion des Séances **Owner** Yosser Rafrafi **Entities** Seance, Lieu	• Create, edit, confirm and cancel photo sessions (wedding, portrait, product, event). • Manage shooting locations (studio room, outdoor, client's place) and their availability. • Assign a photographer to each session and prevent double bookings. • Calendar view of all sessions with status tracking (Planned, Confirmed, Done, Cancelled). • **AI:** suggest the best time slot from photographer and location availability.
Gestion des Photos & Galeries **Owner** Louay Herichi **Entities** Photo, Galerie	• Bulk upload of session photos to cloud storage, with thumbnails and EXIF metadata. • **AI culling:** blur and closed-eye detection, quality score, duplicate grouping. • **AI tagging** and natural-language photo search. • Create private client galleries (link, password, expiry date) and share them. • Clients mark favorites, comment and validate their selection; **AI** recommends similar photos and a visual-story order. • Apply an editing style to selected photos and download the final images.
Gestion du Matériel **Owner** Amel Mediouni **Entities** Materiel, Reservation	• Manage the equipment inventory (cameras, lenses, lighting, tripods) with condition and purchase data. • Reserve equipment for a session and block conflicting reservations. • Record equipment returns and report breakdowns. • Equipment availability calendar. • **AI:** suggest the equipment kit for a session from its type and past sessions.

Module	Required capabilities
Gestion de la Facturation **Owner** Aya Flah **Entities** Devis, Facture	• Create quotes (services, discount, VAT) and send them to the client portal. • The client accepts or refuses the quote online. • Generate deposit and balance invoices from an accepted quote. • Record full or partial payments and track unpaid and overdue invoices. • **AI:** generate a quote from the session type and past pricing; flag invoices likely to be paid late.
Gestion du Portfolio **Owner** Anas Ben Ibrahim **Entities** Portfolio, ElementPortfolio	• Create portfolio collections by category (wedding, portrait, product, event). • Add photos from client galleries, only with the client's consent. • Order, feature and caption portfolio items; publish or unpublish collections. • Public portfolio pages for visitors, with view statistics. • **AI:** select the best photos from quality scores and generate captions.
Gestion des Avis & Réclamations **Owner** Mohamed Yessine Nahdi **Entities** Avis, Reclamation	• Clients rate and review a completed session; the studio can reply and publish reviews. • Clients file complaints (delay, quality, billing) linked to a session. • Staff take charge of complaints, answer them, and resolve or reject them with status tracking. • **AI:** sentiment analysis of reviews; automatic classification and prioritization of complaints.

2.2.NON-FUNCTIONAL REQUIREMENTS

🔐 1. Security

Secure authentication, role-based access, protected APIs/files, input validation, HTTPS, and secure secrets.

⚡ 2. Performance

Fast, responsive UI; large photo uploads; asynchronous AI processing with progress tracking.

📶 3. Availability & Reliability

Clear operation status: Pending → Processing → Completed / Failed.

🔄 4. Scalability

Support growing studios, users, sessions, photos, and AI jobs through modular architecture.

🛠️ 5. Maintainability

Modular code, clear naming, reusable components, API contracts, and documentation.

🛡️ 6.Reliability

Error handling, data protection, job tracking, and recovery from failed operations.

2.3 ACTORS

The people and systems that interact with StudioFlow, and what each of them can do.

Actor	Description	Main access
Administrateur (studio manager)	Runs the studio and configures the platform.	Full access: users and roles, sessions, equipment, billing, portfolio, reviews and complaints.
Photographe	Covers the photo sessions.	Assigned sessions, equipment reservations, photo upload and AI culling, client galleries.
Retoucheur	Edits and finalizes the photos.	Selected photos, editing styles, final delivery of galleries, portfolio items.
Client	Customer of the studio.	Client portal: sessions, quotes and invoices, gallery selection and downloads, reviews and complaints.
Visiteur	Public visitor, not logged in.	Browses the public portfolio and discovers the studio.
Système IA (secondary actor)	AI services called by the modules.	Culling, tagging and search, recommendations, time-slot and quote suggestions, sentiment analysis, complaint classification.

2.4 TEAM ORGANIZATION

Each member owns one business module end to end. The core module and the shared layer are built together.

Member	Module	Entities
Yosser Rafrafi	Gestion des Séances	Seance, Lieu
Louay Herichi	Gestion des Photos & Galeries	Photo, Galerie
Amel Mediouni	Gestion du Matériel	Materiel, Reservation
Aya Flah	Gestion de la Facturation	Devis, Facture
Anas Ben Ibrahim	Gestion du Portfolio	Portfolio, ElementPortfolio
Mohamed Yessine Nahdi	Gestion des Avis & Réclamations	Avis, Reclamation
Whole team	**Gestion des Utilisateurs (core)** + shared layer (auth, UI kit, storage, AI client)	Utilisateur, Notification

Collaboration rules

- One Git branch per module; merges into the main branch go through a pull request reviewed by another member.
- Each module lives in its own folder (modules/<module>) with its models, services, validation schemas and pages.
- A module never reads another module’s collections directly: it calls that module’s service functions.
- Shared code (authentication, UI components, storage, AI client) changes only through a reviewed pull request.

03

GRAPHIC DESIGN CONCEPTION (UI / UX)

🎨 GRAPHIC CHARTER

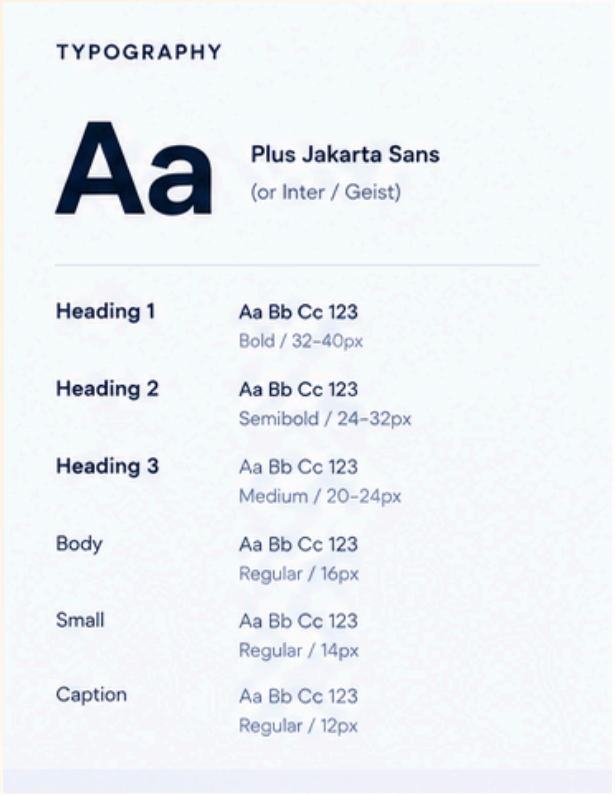
Application Logo :



Group Logo :



🎨 GRAPHIC CHARTER

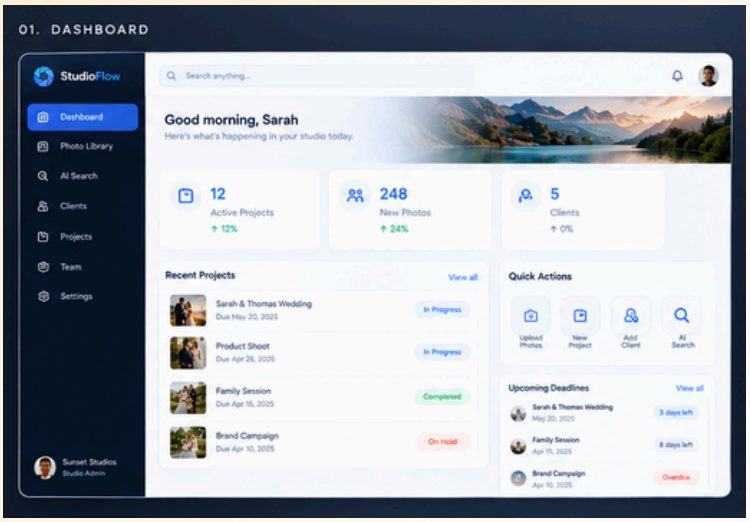




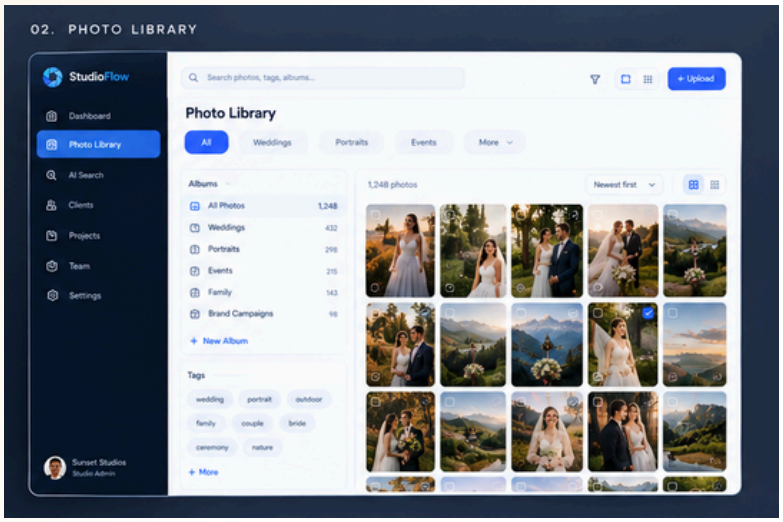
🎨 GRAPHIC CHARTER

UI/UX conception :

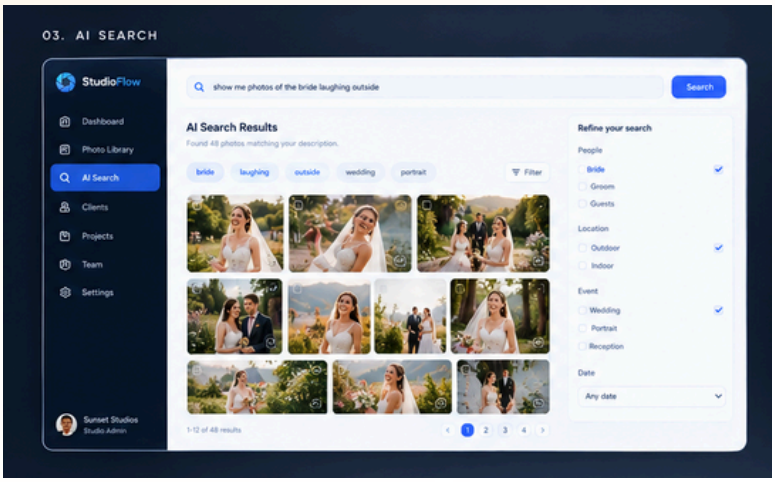
01. DASHBOARD



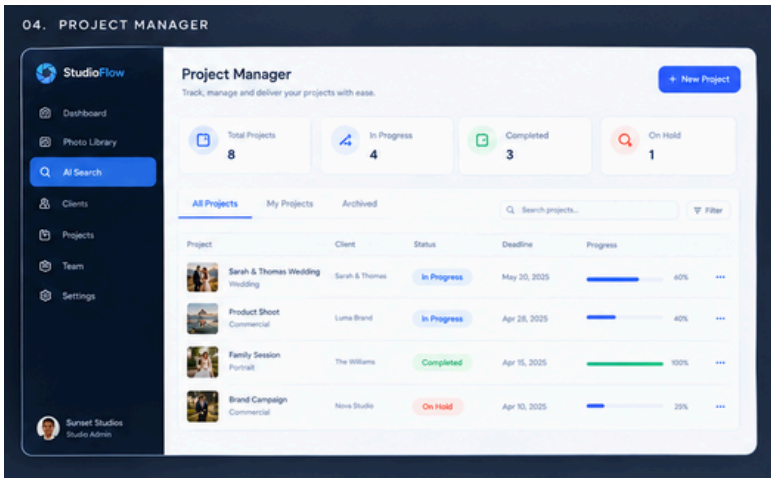
02. PHOTO LIBRARY



03. AI SEARCH



04. PROJECT MANAGER



04

TECHNICAL CONCEPTION & ARCHITECTURE

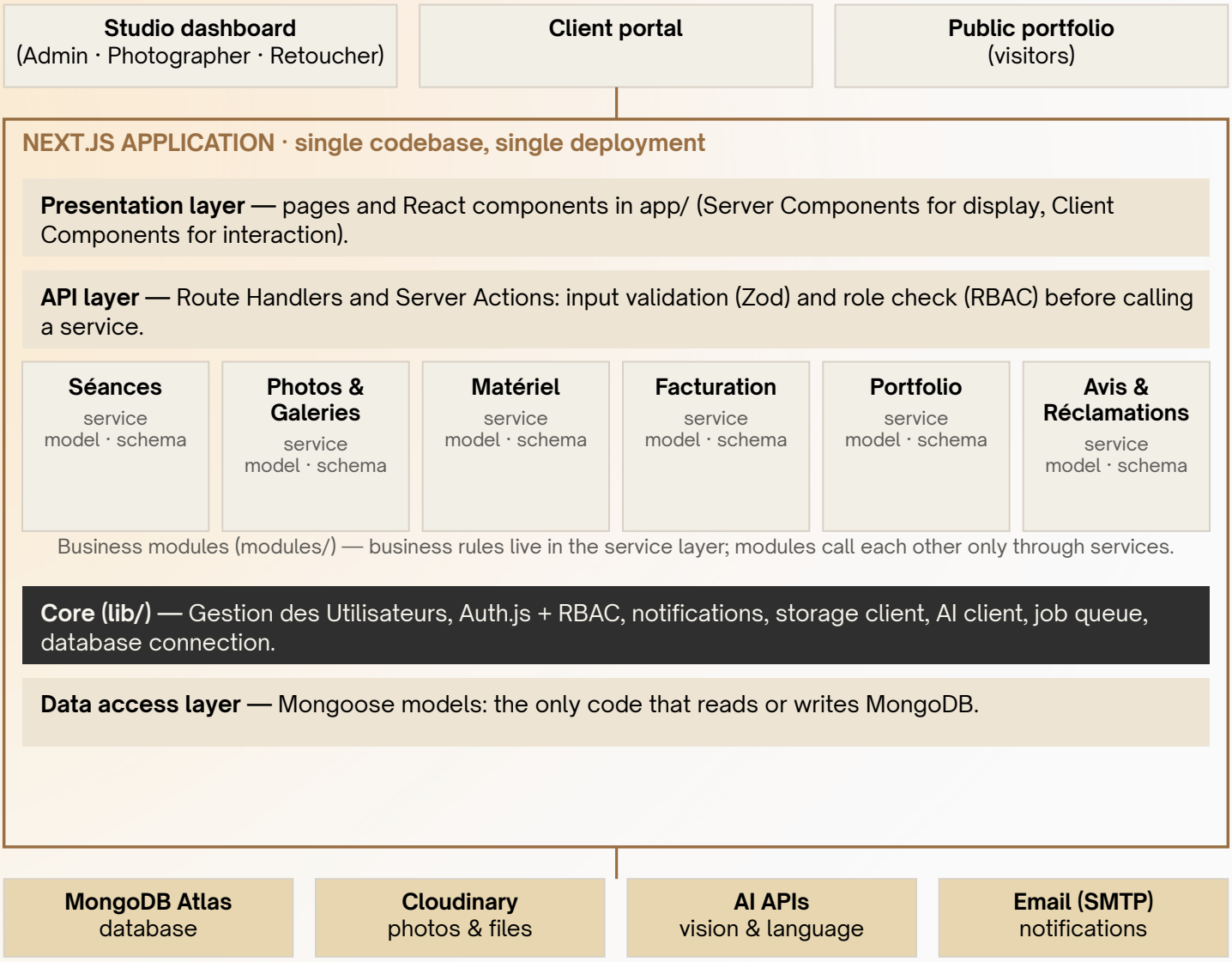
4.1 TECHNOLOGY STACK

StudioFlow is one Next.js application for both the frontend and the backend, backed by a MongoDB database.

Layer	Technology	Usage in StudioFlow
Framework	Next.js (App Router), React, TypeScript	Pages, Server Components, Route Handlers and Server Actions in a single codebase (frontend + backend).
User interface	Tailwind CSS, shadcn/ui	Shared UI kit that follows the graphic charter (colors, Plus Jakarta Sans typography).
Database	MongoDB (MongoDB Atlas)	Document database for all modules; flexible for photo metadata, AI tags and scores.
Data access & validation	Mongoose, Zod	Schemas, enums and references between collections; validation of every input.
Authentication	Auth.js (NextAuth)	Login, sessions, password hashing (bcrypt) and the user role carried in the session (RBAC).
File storage	Cloudinary	Photo and document upload, automatic thumbnails, fast delivery to galleries.
Artificial intelligence	Vision and language model APIs	Culling, tagging, search, recommendations, quote generation, sentiment analysis.
Background jobs	Agenda (job queue on MongoDB)	Long AI tasks run outside the request, with status Pending → Processing → Completed / Failed.
Notifications	In-app notifications + Nodemailer	Alerts for sessions, payments, shared galleries and complaints.
Quality & tooling	Git / GitHub, ESLint, Prettier, Vitest, Playwright	Version control, code style, unit tests per module and end-to-end tests.
Deployment	Vercel, MongoDB Atlas, Docker (local)	Continuous deployment of the Next.js app; a local MongoDB container for development.

4.2 SOFTWARE ARCHITECTURE

Modular monolith with layers. One Next.js application and one deployment, split internally into one module per gestion. Inside each module, code is organized in layers, and each layer only calls the layer below it.



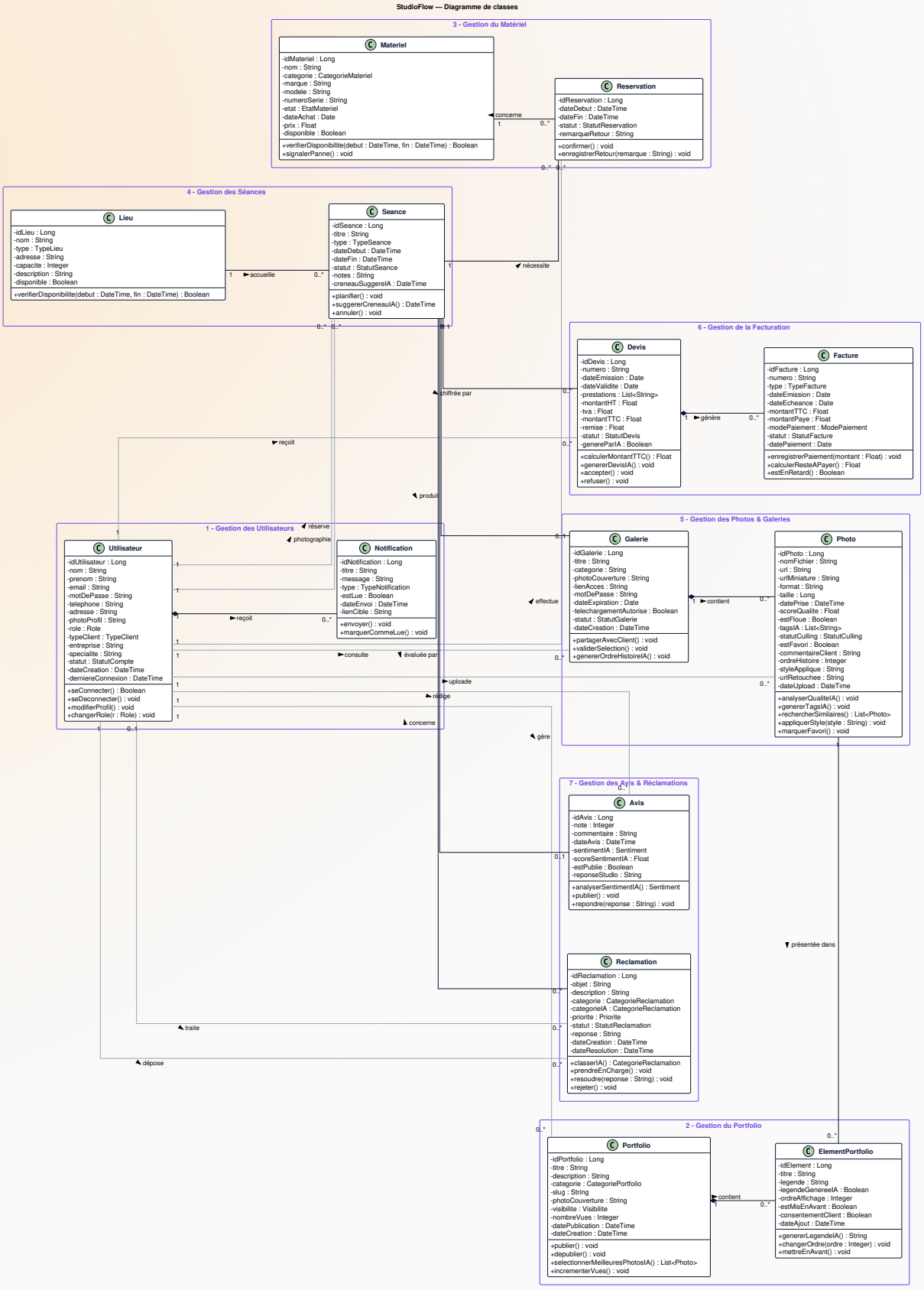

```
src/
├─ app/                  routes: pages + api/
│  ├─ (dashboard)/       studio staff
│  ├─ (client)/          client portal
│  ├─ (public)/portfolio public pages
│  └─ api/               route handlers
├─ modules/              one folder per gestion
│  ├─ seances/  photos-galeries/  materiel/
│  └─ facturation/  portfolio/  avis-reclamations/
├─ lib/                  core: db, auth, ai, storage
└─ components/ui/        shared UI kit
```

Why a modular monolith

- Next.js already serves the frontend and the backend.
- One deployment instead of several services.
- One module per member: clear ownership.
- Module independence through service layers.

4.3 CLASS DIAGRAM

Seven packages for the seven modules. ◆ = composition; other links are associations with their multiplicities. Enumerations are listed in 4.4.



4.4 DATA MODELING (MONGODB)

Each class becomes a collection or an embedded document. Small compositions always read with their parent are embedded; large or independently queried entities get their own collection, linked by ObjectId references.

Class	Storage	Justification
Utilisateur	Collection `utilisateurs`	Referenced by every module (client, photographer, author, staff).
Notification	Collection `notifications`	Grows continuously; always queried per user.
Seance · Lieu	Collections `seances`, `lieux`	Central entity; references the client, the photographer and the location.
Materiel · Reservation	Collections `materiels`, `reservations`	Reservations are searched by date to detect conflicts.
Galerie · Photo	Collections `galeries`, `photos`	Thousands of photos per gallery: too large to embed (16 MB document limit).
Devis · Facture	Collection `devis`, Facture **embedded**	Composition: a few invoices per quote, always read together.
Portfolio · ElementPortfolio	Collection `portfolios`, elements **embedded**	Ordered list read in one query to render a public page.
Avis · Reclamation	Collections `avis`, `reclamations`	Queried and filtered independently (status, sentiment, priority).
Enumerations	String fields	Validated with Mongoose enum (Role, StatutSeance, StatutFacture, Sentiment...).

Enumerations

Role
`ADMIN · PHOTOGRAPHE · RETOUCHEUR · CLIENT`

StatutSeance
`PLANIFIEE · CONFIRMEE · TERMINEE · ANNULEE`

StatutGalerie
`EN_TRI · PARTAGEE · VALIDEE · LIVREE`

StatutCulling
`RETENUE · REJETEE · EN_ATTENTE`

StatutDevis
`BROUILLON · ENVOYE · ACCEPTE · REFUSE · EXPIRE`

StatutFacture
`EN_ATTENTE · PARTIELLEMENT_PAYEE · PAYEE · EN_RETARD`

TypeFacture
`ACOMPTE · SOLDE`

CategoriePortfolio
`MARIAGE · PORTRAIT · PRODUIT · EVENEMENT`

Visibilite
`PUBLIC · PRIVE`

Sentiment
`POSITIF · NEUTRE · NEGATIF`

StatutReclamation
`OUVERTE · EN_COURS · RESOLUE · REJETEE`

CategorieReclamation
`RETARD · QUALITE · FACTURATION · AUTRE`